Oryx

The fastest, most beautiful markdown and code viewer on the planet.

Oryx started as a personal need: reading a markdown file should not require opening an editor, a browser tab, or an Electron app. Most tools either edit with a preview attached or embed a web engine to draw text. I work with a lot of markdown and rarely edit it, so I wanted speed and convenience. Oryx renders markdown natively, in a single small binary that draws everything itself on the CPU, with the typography and theming a reader deserves, nothing else.

Install · Use · Themes

Why Oryx

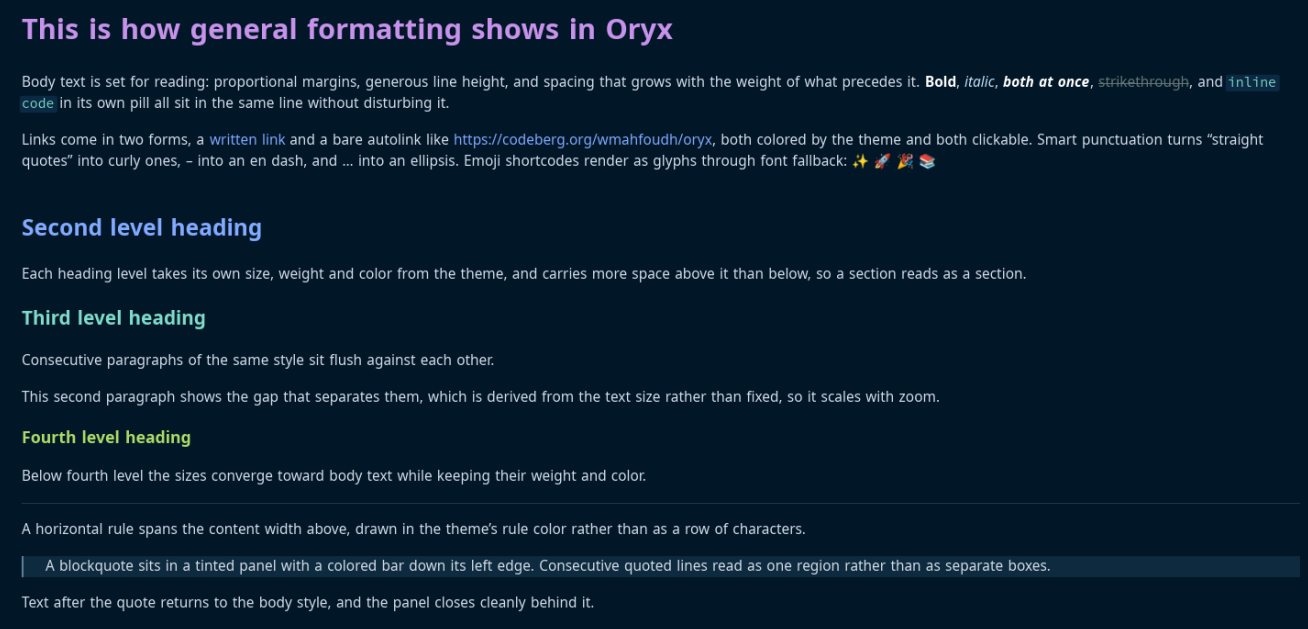
It opens instantly, whatever the size. A normal document is on screen in well under 150 milliseconds from cold, and an 8MB file takes about a third of a second. Oryx manages that by highlighting and laying out only the first few screens before it paints, then finishing the rest in the background while you read.

It reads, it does not edit. There are no panes and no toolbars. F1 lists every shortcut and Escape closes whatever is open.

It is one binary and a folder of themes. Every dependency is pure Rust, so there is nothing to install beside it and no browser engine hiding inside. It behaves the same on a new laptop as on an old machine with no GPU.

What it shows

Formatting. The picture below covers most of it: headings, bold, italic, strikethrough, inline code, links and bare URLs, smart quotes and dashes, rules, nested blockquotes, and emoji shortcodes like `:tada:`.



Code. Fenced blocks get a bordered panel and syntax colors for the languages most people write. A line too long for the panel wraps inside it. You can also point Oryx straight at a source file and it renders the whole thing as one highlighted document. Close to a hundred extensions carry colors, from Rust and Python through Haskell, Scala, LaTeX, Makefiles and diffs. Anything else holding text still opens in the code font, so a `Makefile` or a `.conf` reads cleanly without them. Hand Oryx a binary and it says so in one line.

This is Oryx rendering highlighted code

Fenced blocks sit in a bordered panel, one row per source line, with keywords, strings, numbers, comments and operators each taking their color from the active theme rather than from the language.

```
/// Opens a file and returns the parsed document.
pub fn open(path: &Path, deadline: Option<Instant>) -> Result<Opened> {
    let bytes = std::fs::read(path)?;
    let text = String::from_utf8_lossy(&bytes);
    let mut document = match detect(path) {
        FileKind::Markdown => markdown::parse(&text),
        FileKind::Code(token) => code_document(token, &text),
        FileKind::Plain => plain_document(&text),
    };
    let pending = apply_budget(&mut document, deadline);
    Ok(Opened { document, pending })
}
```

```
def summarize(rows, key="total"):
    """Group rows and total one column."""
    out = {}
    for row in rows:
        out[row.name] = out.get(row.name, 0) + row[key]
    return sorted(out.items(), key=lambda kv: -kv[1])
```

```
# Shell, with strings, comments and variables colored
for file in "$@"; do
    printf 'rendering %s\n' "$file"
    oryx --theme nord "$file" || echo "failed: $file" >&2
done
```

A line longer than the panel wraps inside it rather than spilling out or being cut off:

```
const themes = ["oryx-light", "oryx-dark", "dracula", "nord", "gruvbox-dark", "catppuccin-mocha", "tokyo-night", "solarized-light", "everforest-dark"];
```

Code with no language, or a language Oryx does not know, still gets the panel and the monospace family, in the plain code color:

```
$ oryx README.md
opened in 41ms
```

Tables. Columns keep the alignment you gave them, rows alternate their background, and each column sizes itself to its content up to a limit. A cell with a lot of text wraps inside its column, so a wide table never runs off the page.

This is Oryx rendering tables

Columns size themselves to their content up to a cap, rows alternate their background, and the header spans in bold above a grid drawn in the theme's own colors.

Shortcut	Action	Notes
Ctrl+O	Open file	Native dialog, filtered to what Oryx reads
Ctrl+T	Theme browser	Previews and applies live
Ctrl+B	Folder sidebar	Keyboard navigable
Ctrl+F	Find in document	Smart case matching
F1	Shortcuts help	Every binding, in one screen

Alignment markers in the separator row are honored per column, left, center and right:

Language	Extension	Files
Rust	`.rs`	1284
Python	`.py`	96
TypeScript	`.ts`	12
Markdown	`.md`	7

Cells carry inline styling of their own, and long cells wrap inside their column instead of pushing the table off the page:

Element	Behavior
Bold cell	Styling inside a cell renders exactly as it does in a paragraph
`code cell`	Inline code keeps its pill and its monospace family
Link cell	Links stay clickable inside the grid
Wrapping cell	A deliberately long cell, written to run past the column width so the wrapping inside the cell is visible and the row grows to fit it rather than overflowing

A compact table stays narrow rather than stretching to the full content width:

Yes	No
1	0

Lists. Ordered, unordered, nested as deep as you like. A wrapped line lines up with the text above it, not with the bullet. Task lists get real checkboxes.

This is Oryx rendering lists and task lists

Unordered lists take a bullet sized to the text, and nesting indents by a fixed step so the levels line up down the page.

- A first item, at the top level
- A second item, long enough to wrap onto a second line so the continuation aligns under the text rather than under the bullet
  - A nested item, one level in
  - Another nested item
    - A third level, indenting again
- Back at the top level

Ordered lists number themselves from the source, and the marker is laid out as its own run so the text after it aligns regardless of the number width.

1. The first step
2. The second step
3. The third step, which wraps the same way an unordered item does and keeps its continuation aligned with the text
   1. A nested ordered item
   2. And a second one
4. A two-digit marker, still aligned

Task lists draw real checkboxes, filled when checked, in the theme's colors.

- [x] Render the document natively
- [x] Draw every pixel on the CPU
- [x] Ship as one small binary
- [ ] Embed a browser engine
- [ ] Require a runtime

Lists sit tighter against each other than paragraphs do: consecutive items take a small gap, while the space before and after the whole list matches the surrounding block spacing.

- Tight against the item above
- And the one below

Alerts. All five GitHub alert kinds are styled: note, tip, important, warning and caution, each with its own color and title.

This is Oryx rendering alerts and blockquotes

GitHub alerts take a titled panel with a colored bar and a matching title, one color per kind, all drawn from the active theme.

> **Note**
> Useful information that a reader should notice even when skimming.

> **Tip**
> An optional suggestion that makes something easier.

> **Important**
> Information a reader needs in order to succeed at the task.

> **Warning**
> Something that needs immediate attention because of the risk it carries.

> **Caution**
> The consequences of an action that cannot easily be undone.

Plain blockquotes

A quote without a marker takes the neutral panel and bar:

> The best way to predict the future is to invent it. A quote can run to several lines, and the panel closes around all of them as one region rather than boxing each line separately.

Quotes nest, and each level takes its own indent and its own bar, so the depth is readable at a glance:

> A first level quote, introducing what follows.
>
> > A second level, quoted inside the first.
> >
> > > And a third level, as deep as most documents ever go.

Quoted content keeps its own formatting, including **bold**, `code`, and lists:

> Inside a quote you still get:
>
> - list items with their bullets
> - `inline code` in its pill
> - links that stay clickable

Frontmatter. A YAML header, the kind Obsidian and static site generators put at the top of a file, becomes a small metadata panel above the document. Most viewers either print it as a paragraph or throw it away.

```yaml
title: Field notes on the Arabian oryx
author: W. Mahfoudh
date: 2026-07-26
status: published
tags: reading, markdown, typography
draft: false
```

This is Oryx rendering YAML frontmatter

The block above the title is YAML frontmatter, the metadata header that static site generators, note-taking apps and documentation systems put at the top of a file. Most viewers either print it as raw text or hide it entirely.

Oryx renders it as a metadata panel: a tinted block above everything else in the document, each key and value on its own line, in the theme's frontmatter colors rather than the body colors, so it reads as a header rather than as the first paragraph.

The panel always sits first, whatever follows it. Ordinary content resumes underneath at the normal body style, as this paragraph does, with the usual spacing between the panel and the first heading.

Why it matters

A note file that starts with eight lines of metadata is unreadable when those lines are printed as a paragraph, and incomplete when they are thrown away. Keeping them, visibly separated, is the only treatment that respects both the file and the reader.

Math. Math written as TeX comes out with real symbols: `\sum` is $\sum$, `\alpha` is α, and superscripts and subscripts sit where they should, whether the expression is inline in a sentence or centered on a line of its own.

This is Oryx rendering math

Math is written as TeX literals between dollar signs. Oryx substitutes the symbols and raises the scripts, so an expression reads as an expression rather than as source: the mass-energy relation $E = mc^2$, a tolerance of $\varepsilon \le 0.01$, and a sum $\sum_{i=1}^{n} x_i^2$ all sit inline in the paragraph at the surrounding text size.

Block math is centered on its own line:

$$\sum_{i=1}^{n} (x_i - \mu)^2 \le \sigma^2 \cdot n$$

$$\forall\ \varepsilon > 0,\ \exists\ \delta > 0 : |x - a| < \delta \Rightarrow |f(x) - f(a)| < \varepsilon$$

$$\int_0^{\infty} e^{-x^2}\,dx = \sqrt{\pi} / 2$$

Greek letters and operators

Lowercase $\alpha\ \beta\ \gamma\ \delta\ \varepsilon\ \theta\ \lambda\ \mu\ \pi$
$\sigma\ \varphi\ \omega$ and uppercase $\Gamma\ \Delta\ \Theta\ \Lambda\ \Pi\ \Sigma$
$\Phi\ \Psi\ \Omega$ come through as their own glyphs.

So do the operators you actually reach for: $a \times b$, $a \cdot b$, $a \div b$, $x \pm y$, $a \neq b$, $a \approx b$, $a \equiv b$, $a \propto b$, ∇f, ∂x, and ∞.

Set and logic notation reads the same way: $x \in A$, $y \notin B$, $A \subseteq B$, $A \cup B$, $A \cap B$, $A \to B$, $p \wedge q$, $p \vee q$, $\neg p$, and the empty set $\varnothing$.

Scripts

Superscripts and subscripts bind the next character or a braced group, exactly as in TeX: x^2, a_i, x^{n+1}, $a_{i,j}$, and both at once in x_i^2. Nested cases like σ_{max}^2 keep their grouping.

Footnotes. Markers sit raised in the text and jump to their definitions, which gather at the foot of the document under a rule, in the order they were referenced.

This is Oryx rendering footnotes

Footnote references render as raised superscript markers in the flow of the sentence[why], sized down from the body text so they mark without interrupting. Clicking one jumps to its definition.

Definitions collect at the end of the document under a rule, in reference order, wherever they were written in the source[order]. The one below was written first in the file and still lands in its proper place.

A paragraph can carry several notes at once[one] without the line spacing shifting around them[two], because a raised marker is laid out inside the line box rather than above it[three].

Notes can be as long as they need to be, wrapping across several lines at a slightly smaller size than the body so the block reads as apparatus rather than as prose[long].

[order] Written first in the source, placed third here, because definitions are ordered by the references that point at them.

[why] The jump is a real anchor: Oryx resolves it against the laid-out document, so it lands exactly on the definition.

[one] The first of three in one paragraph.

[two] The second. Markers stay on their own baseline.

[three] The third, which proves the line spacing above it did not move.

[long] A longer note, to show how a definition wraps. It runs past a single line so the indentation and the reduced size are both visible, and it ends without any of the surrounding text being pushed around.

Images and badges. Local images render, PNG or SVG. Remote ones are fetched in the background and cached on disk, so a README covered in shields.io badges comes up immediately the second time you open it, and keeps working offline. If a path is broken you get a placeholder with the alt text in it.

This is Oryx rendering images

Local raster images are decoded and scaled to the content width, keeping their aspect ratio, and an image smaller than the text column keeps its natural size instead of being stretched.

SVG images are rasterized at their intrinsic size when the document opens, so vector art stays crisp instead of being blown up from a bitmap.

GitHub READMEs. Real READMEs lean on raw HTML for the things markdown cannot do, so Oryx handles the common subset: centered blocks, images at a set width or height, rows of clickable badges, line breaks, and the inline tags down to sub and sup.

This is Oryx rendering a GitHub style README

Real project READMEs are not plain markdown. They open with a centered logo, a wrapped row of badges, and a tagline built from raw HTML, because markdown alone cannot center anything. Oryx renders that subset directly, so a README looks the way its author intended.

A fast, native markdown viewer
Reading, not editing
One binary, no runtime

What the HTML subset covers

Centered blocks through `<p align="center">` and `<div align="center">`, images with an explicit `width` or `height`, images wrapped in links so a badge stays clickable, `<br>` as a hard line break, and the inline styling tags: **bold**, *italic*, `code`, H_2O with a subscript and 10^6 with a superscript.

Badges also sit inline inside ordinary text, at the size the tag asks for: the project is

and runs on

.

What it does not cover

HTML tables and `<details>` sections are not rendered. Any other tag is stripped and its inner text kept, so an unsupported tag costs its styling and never its content.

Find in document. Ctrl+F searches, and the search is smart about case: type `oryx` and you match Oryx, ORYX and oryx; add a capital and only Oryx matches. It looks everywhere text is laid out, so list items, link text, table cells, quotes and code blocks all count. A match can run across styling, so `fast viewer` turns up even when it was written as **fast** viewer, though it will not run from one block into the next.

Interface. A folder sidebar you can drive from the keyboard, showing a type icon beside each file. A native open dialog. Selection with copy as plain text or as the original markdown. Per-session zoom, reload from disk for files you are editing elsewhere, and the shortcuts screen on F1. Window geometry, the sidebar and the folder you were last in are remembered between runs.

Install

From a release

Download the archive for your platform from the releases page, extract it, and run the installer inside:

```
tar -xzf oryx-*-linux-x86_64.tar.gz && cd oryx && ./install.sh
```

On Windows, extract the zip and run `install.ps1` in PowerShell. The installer copies the binary and themes, and registers the file association so markdown files open with Oryx from your file manager. `./install.sh --uninstall` removes everything.

From source

Requires Rust 1.80 or later.

```
git clone https://codeberg.org/wmahfoudh/oryx.git
cd oryx
make install
```

`make install` builds the release binary, installs it to `~/.local/bin`, copies the themes to `~/.local/share/oryx/themes`, and registers the file association. Plain `cargo build --release` works too; the binary looks for `themes/` next to itself, in the XDG data directory, and in the working directory. For everyday reading use the installed binary or `--release`: a plain debug build is noticeably slower on code-heavy documents.

Use

> There are no menus. **Press F1** for the complete shortcut list, Escape to close a panel or quit.

```
oryx README.md          open a file
oryx src/main.rs        code files render highlighted
oryx --theme nord file  pick a theme for this session
oryx --register         install the file association and icons
oryx --version          print the version
```

Shortcut	Action
Ctrl+O	Open file
Ctrl+,	Settings

Ctrl+T	Theme browser
Ctrl+B	Folder sidebar
F1	Shortcuts help
F5 / Ctrl+R	Reload from disk
Ctrl+Plus / Ctrl+Minus	Zoom in / out
Ctrl+0	Reset zoom
Ctrl+A	Select all
Ctrl+C	Copy selection as text
Ctrl+Shift+C	Copy selection as markdown
Ctrl+F	Find in document
F3 / Shift+F3	Next / previous match
Up / Down	Scroll by line
Page Up / Page Down, Space / Shift+Space	Scroll by page
Home / End	Jump to top / bottom
Escape	Close overlay or sidebar, quit

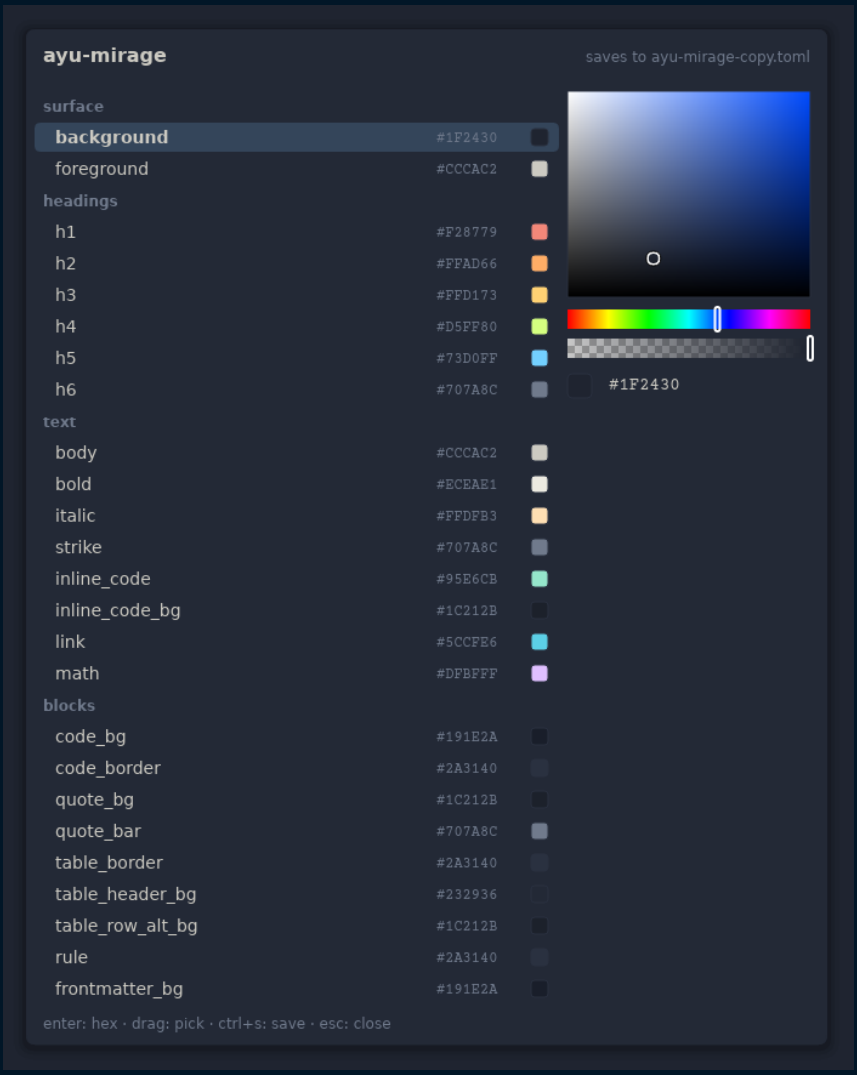
Ctrl is Cmd on macOS. Copy as markdown reproduces the original source of the selection, styles intact.

Themes

Thirty-one themes ship with Oryx. Each one is a single TOML file with 51 color roles, so every element can be colored on its own. Leave a key out and it falls back to a default. Write something malformed and Oryx skips the file and keeps the theme it already had.

The browser on Ctrl+T previews them and applies them live.

The editor changes any role with a color picker while the document restyles behind it. Edit one of the bundled themes and Oryx quietly edits a copy, so the files it shipped with stay as they were.



Nine are original designs: `oryx-light` (the default) and its dark twin `oryx-dark`, `oryx-sand` and `oryx-night`, `inkstone`, `ember`, `meadow`, `slate`, and `be-vendible`. The rest adapt permissively licensed editor palettes, credited below.

Performance

Oryx works in four stages: load, layout, paint, present. It parses the file when you open it, lays the blocks out in order, and paints a band that covers the viewport and a couple of screens either side. Scrolling inside that band is a memory copy, which is why the cost of a scroll frame has nothing to do with how long the document is. The event loop wakes only for input, so an idle window uses no CPU at all.

Both of the expensive stages are given a time budget rather than being allowed to block, and that is what makes a large file feel like a small one.

Syntax highlighting. Opening a file spends about forty milliseconds coloring code, and a background thread sends the rest along in chunks. Each chunk recolors lines that have already been laid out, so nothing moves on screen.

Text layout. Opening lays out the first screens and the rest follows in order, a slice at a time, between frames. A position is exact as soon as it exists and never shifts afterwards, so a selection you make while the document is still filling stays where you put it. Zoom, the sidebar and window resizing all run through the same machinery. The scrollbar tracks what has been laid out so far.

Measured on one Linux machine, release build, from launch to the first frame. The eager column is the same two stages with their budgets removed, the way earlier versions worked.

Document	Eager	Budgeted
1MB source file	4.6s	81ms
8MB source file	38s	90ms
1MB markdown	1.8s	82ms
8MB markdown	14s	317ms

Opening a source file is effectively constant time at any size.

A performance test in the repository checks the startup, relayout and paint timings.

Limits

Oryx is built for everyday documents, and there are things it will not do.

- It does not edit files.
- Math is drawn as styled text with real symbols, not properly typeset.
- Open something several megabytes long and the colors, and the layout below the first screens, take a moment to catch up.
- Memory grows with the file. An 8MB document costs a few hundred megabytes while it is open.
- The HTML it understands is a deliberate subset. No HTML tables, no collapsible sections.
- macOS compiles, but it is untested and there is no packaged build.

Some of these are on the list for future versions; none of them are promises.

Under the hood

The layout engine draws everything, which means the rendering can be tested as numbers rather than by eye. More than 250 tests assert positions, wrapping, spacing and colors. Every color on screen comes from the active theme file. Every dependency is pure Rust, and that is what keeps the binary small, the startup quick and the build straightforward on all three platforms.

The files under `tests/showcase/` are the documents behind the screenshots above. Each one exercises a single feature, and they are worth opening to see what Oryx makes of them.

Bug reports and theme contributions are welcome on the issue tracker. Pull requests are read with interest, though without promises. `make check` is the gate: formatting, clippy for the Linux, Windows and macOS targets, build, and the full test suite.

Credits

DejaVu Sans and Courier Prime are embedded in the binary. DejaVu is distributed under the DejaVu Fonts License, Courier Prime under the SIL Open Font License. The settings dialog can switch to any family installed on your system.

The adapted themes come from these palettes, all MIT, with thanks to their authors:

- Dracula (draculatheme.com)
- Nord (nordtheme.com)
- Gruvbox dark and light (morhetz/gruvbox)
- Catppuccin Mocha and Latte (catppuccin.com)
- Tokyo Night (enkia/tokyo-night-vscode-theme)
- Solarized dark and light by Ethan Schoonover (ethanschoonover.com/solarized)
- One Dark (atom)
- Everforest dark and light (sainnhe/everforest)
- Rosé Pine and Rosé Pine Dawn (rosepinetheme.com)
- Kanagawa (rebelot/kanagawa.nvim)
- Ayu Mirage and Light (ayu-theme)
- Night Owl by Sarah Drasner (sdras/night-owl-vscode-theme)
- Horizon (jolaleye/horizon-theme-vscode)
- Flexoki dark and light by Steph Ango (stephango.com/flexoki)
- GitHub Light (primer/primitives)

License